FAQ: GVM

Graph Virtual Machine (GVM): Naming, Topology, Equations

DAEDAELUS (DAE) has developed a low-cost, high-performance Transaction Fabrix^{TM} for datacenters using N2N (Neighbor-to-Neighbor) links (repurposing Ethernet frames), with selectable (reliable, treecast) delivery based on a new way to rendezvous on *trees* (without fixed IP endpoint names). The lowest level graph is a set of black trees,where each black tree in the set is named by the cell on which they are rooted; together they form our *groundplane*. The architecture is layered upwards through logical stacked trees that provide hardware-enforced *confinement*, and virtual stacked trees that provide a foundation on which developers can automatically provision microservice graphs with *an equation*.

Introduction

The DAEDAELUS (DAE) Transaction Fabrix (TF) uses standard Ethernet NICs. However, East-West interactions use tree-based naming for address groups of closely related microservices in *sets*, rather than the source/destination addressing mode that practitioners of conventional networks are accustomed to[1].

This simple change, implemented *below* L2, is the first step to enabling *Structured Topology Management* of microservice sets that need to evolve dynamically in response to perturbations (failures, disasters, attacks).

With 8 ports per cell, a 1-hop cluzter comprises 9 cells. The center cell is the root of its own tree, and the default transaction manager for consensus operations. Neighbor cells become proxies for resource (compute/memory/storage) elasticity, dynamic load balancing and failover. A preselected neighbor is provided for voluntary (or involuntary) failover when the center cell dies or needs to be taken out of service.

Trees extend to any number of hops, out to the edge of the Transaction Fabrix (the boundary to the conventional network). The primary means of addressing is *radial* (by port) - it knows which 'direction' a target entity exists along, but may not know 'how far' along a branch, or which sub-branch it exists. This is an essential mechanism – to allow the migration of an entity without having to change addresses from the point of view of the source requestor. This mechanism also enables failover and elastic growing and shrinking of a microservice set in response to traffic demand.

Trees are subsetted to enable hardware confinement between zones. These are used typically to create tenants and subtenants within a datacenter. All these API's are directly available to the distributed systems developer through an API. Within the Transaction Fabrix, there is no need for conventional datacenter segmentation using firewalls, iptables, or ACL's in the routers.

The central mechanism driving this simple yet powerful approach to datacenter address management is the Graph Virtual Machine (GVM): A protected virtual machine environment running in the FPGA Substructure (SmartNICs). All packets going into a cell go through this vantage point enables: routing to be done differently, provisioning to be done differently, and security to be done differently.

[1]This FAQ assumes that the reader has read the first two 'read-me-first' documents: Exec-Summary-Links (Rethinking Datacenter Fabrix) and Exec-Summary-TRAPH.pdf (Rethinking Datacenter Management)

Fundamental organizing unit: Graphs, not Lists

The entire datacenter is a graph (groundplane). All (non-trivial) applications in datacenters are sub-graphs. We create sub-graphs by recursively stacking trees (along with their failover links) above the groundplane. These sub-graphs are the regions or tenants that respond to commands from an orchestrator to do something. Within each subtenant, developers can orchestrate their own microservice sets without being required to work within the constraints of conventional networking tools, such as command line interfaces and ACL's in switches/routers, or the cumbersome sets of rules used in todays internal segregation firewalls.

It is inevitable that the above description will create more questions than it answers.
The rest of this document is a deeper dive for the technically curious.

This FAQ is one of many in the DAEDAELUS repository.

List of Questions

[**Q1**]What problem is the DAEDAELUS Graph Virtual Machine (GVM) solving?

[**A1**] The GVM addresses the compounding complexity of naming and address management; perhaps the most pernicious problem in today's datacenters. This root cause of fragility and vulnerability, leads to serious impediments to business velocity in today's Cloud Computing infrastructures. This will only get worse as the cloud distributes and extends AI Infrastructures to the 'edge'.

[**Q2**]Can the GVM be made to work over conventional Clos networks?

[**A2**] No. The nature of communication in GVM isn't a packet fired to a destination address and then forgotten, it's an 'intimate' connection between two communicating entities, able to not only carry messages between those entities but also maintain shared data structures whose contents on all entities on the tree remain consistent. The GVM also relies on the underlying Transaction Fabrix to *successively reverse* the protocol in for a failed message to return all entities to the prior

agreed on state.

[**A2**] The nature of a named tree is that not only can its path be locally rerouted around a communication failure, without packet loss, but also the target members within the set named by that `treeID` can be locally reassigned to another target in the same set, in the event of node failure, without needing to notify or update any of the elements communicating with or through the failed node.

[**A2**] The current art with Clos networks allows packets to be dropped in the event of link failure, bit error, congestion of various kinds, and certain corner case conditions in packet handling from sending through switching to receiving. In the case of link failure, a software path must be invoked to rewrite forwarding tables to route around the link, during which time thousands or even millions of packets could disappear.

[**A2**] While TCP-like timeout and retry can in theory eventually get every packet to its destination in order,. In practice: (1) the nodes of a distributed database don't know what messages they haven't gotten, and in a TCP retry storm could easily diverge in their understanding of basic membership and state. (2) corner cases created by rewriting forwarding tables with packets in flight are intractable, and create transient communication behavior the database will have to be resilient to, even though they couldn't be anticipated.

[**A2**] The current art with IP communication within a data center is that the IP address represents the location of a physical device, not an identity. This is also true of using subnets, or more generally route summarization, as a basic of building IP forwarding tables. The use of IP addresses for both location and identity, and providing IP addresses for both, has been debated and experimented with for decades in the IP community, including the design of IPv6.

[**A2**] Moving a microservice instance from one server to another involves assigning a new IP address, and then telling all of the users of that microservice about the new IP. The nature of GVM is that a set of service instances is given a single name (a `treeid`) and it is only that identity which is known to the callers of that service. The `treeid` can be migrated to a different set of servers without notifying or changing anything at the nodes calling it. (Obviously this move could only be within a single fault containment zone of a single data center, because the connectedness of GVM is lost if forwarded or proxied over a conventional network.)

[**A2**] We have found that by managing infrastructures as 'graphs' instead of 'lists' we not only change the exponent of complexity as an infrastructure grows, we dramatically improve the cognitive clarity of how microservice topologies can managed under developer control, and enable a powerful mechanism for confinement, which creates an entirely new approaches to network security.

[**A2**] Our insight is that these advantages can be gained, not by adopting the archaic architectures, protocols and management tools of the networking industry and forcing them on programmers, but by adopting the programming languages, methodologies and tools of the distributed systems programming community; and extending them to build, manage and tear down the *topologies* of communication between microservices.

[**Q3**]What does the GVM compete with?

[**A3**] GVM competes with YAML files. The difference is that the GVM works algorithmically from a LOV (Local Observer View) vantage point (on the cells controlled by a developer API), where

the structure grows and shrinks, and is owned, by the cells that created them under a developer controlled API. The reason it works algorithmically is that our trees are LOV that grows and shrinks and is owned by the cells. This can only be done using the relative addressing mode of an address-free protocol.

[**Q4**]What's wrong with YAML files?

> It's pretty simple really. The problem with YAML (nice as it is for what it does) is that it has no concept of scale. There are no "subroutines", not even a GOTO or a pointer reference. Stuck at one scale, it fails quickly to adapt as systems grow. Mark Burgess

---

[**Q5**]Can the GVM use conventional IP/port networking?

[**A5**] We believe not. Conventional IP/port networking uses fixed Any-to-Any (A2A) addresses, where IP addresses are tied to physical machines. If you move the execution entity (e.g. container) from one machine to another (even with Kubernetes), you will need to change the IP addresses to be compatible with the new destination. This 'conflation' of name, address and physical location is one of the confounding aspects of the Internet Protocol, which leads to endless complexity in operation.

[**A5**] The DAEDAELUS Neighbor to Neighbor (N2N) addressing is different. Target entities retain their identity (name: `treeID`), and are continually addressable, no matter where they migrate to (or the set expands or shrinks due to failures or dynamic reconfigurations) on the Transaction Fabrix.

[**Q6**]What is the basic principle behind the 'Address-Free' Addressing Scheme?

[**A6**] The basic principle is this: each individual cell is the center of its own universe. All it needs to know is which 'direction' to go to find the related entities in a microservice or lambda 'set'; for example, replicas, or shards, for a Key-Value store. Destination addresses are replaced by 'direction' indicators. This way, replicas (execution entities along with their local state) can migrate back and forth along the branches of a tree, without needing to change the location information; the direction indicators simply guide any packet along the branches of the tree.

[**Q7**]How does the GVM relate to Kubernetes?

[**A7**] Kubernetes is renowned for its complexity. Our contention is that the majority of the confounding complexity of Kubernetes (and for containers in general) is caused by the address management problem described above. It is particularly Challenging for 5G. For example, AT&T is trying to "bring together OpenStack and Kubernetes to simplify network operations".

[**A7**] Network Operations is the primary challenge, but Kubernetes has inherited the challenges and limitations of L3 address management. What we have discovered, is that this problem is vastly harder to address 'on top' of Kubernetes, than it is 'underneath' Kubernetes, with the DAEDAELUS technology.

[**A7**] But providing these TRAPH (TRee-grAPH) based microservice topologies, while a simple and elegant technical solution to the problem, presents a business problem that must be solved for our business to be successful. Market adoption.

[Q8]How do you deal with Legacy/Heritage compatibility necessary for market adoption?

[A8] We provide an IP compatibility mode. If customers demand compatibility, they can have it, but they need to own the cost and complexity that comes with that, not us. Our magic is being able to do far more, far more easily, than the Legacy/Heritage methods can do, while providing a far simpler and more powerful tools to manage topologies under API control by developers.

[A8] We can only do that with a 'clean virtualization' of the network. We don't mix the old and the new in the same protocols on the wire. We use our new DAEDAELUS protocols in an equivalent role to the "Virtual Machine Monitor", pushing all legacy compatibility in to the equivalent role of a Virtual Machine. In this way, our customers can have their cake and eat it. And similar to the magic that VMWare achieved for compute, we don't need the permission of the incumbents to do it.

[Q9]How does the GVM relate to Composable Infrastructure?

[A9] Although the key mechanisms are at the lowest layer in the protocol stack (below L2), GVM equations are composable at a higher level than standard industry definitions from SDX Central and HPE. In order to eliminate the exponential complexity of *diverse* system elements, each cell is fully composed within itself for CPU, memory, storage (persistent memory) *and* networking resources. These complete and self-sufficient 'autonomous agents' are then free to relate to each other directly, for example, to perform self-healing and self-organizing behavior, without the need for dependence on external supervisory or human systems. This kind of 'cell composability' is different from 'component composability' – and as we will see, the difference has far-reaching implications. See FAQ-Latency, FAQ-Bandwidth, FAQ-Resilience, FAQ-Simplicity, FAQ-Composable in our FAQ repository (menu at end).

[Q10]How does the DAEDAELUS form of composability address latency?

[A10] This is *not* an apples for apples comparison. A conventional analysis must take into account the software overhead of processing each packet in the endpoints, and in a conventional network stack, this can be substantial (will overwhelm the simple latency implied here for the link). However, we 'begin' by assuming application layer to application layer latency in software, using DPDK, and PMD. The DAEDAELUS implementation goes several levels deeper in eliminating unnecessary latency: Packet headers are eliminated (see FAQ-Addressing), PCIe overheads are minimized (see: FAQ-Latency), and 'common knowledge' is managed in the link to dramatically reduce the performance impact of multiple round-trip 'coordination' to ensure transactional integrity.

[A10] However, recall that the DAEDAELUS link manages 'common knowledge' on behalf of applications, This enables bypass of the 'PCIe tax' (of 900ns) by exploiting NIC to NIC operations in some phases of multiple (2 or 3) phase commit. More importantly, it enables 0 (zero) phase commit, which masquerades as 'fire and forget' to applications, while maintaining the necessary 'common knowledge' for recovery in the link, until the token is discharged outside the 'reversibility zone'. Thus, we feel entitled to claim lower latencies than normal protocols in conventional switched networks. We could implement this protocol through switches, but *only* if our software was on both sides of the link.

[**Q11**]Why is GVM important to Datacenter Programmability?

[**A11**] Two types of teams co-evolved to manage modern datacenters: one to design and manage the networks, and one to program and manage the servers. This worked when datacenters had a single owner or tenant, their applications and physical infrastructure evolved slowly, and different business units could work within their own silo's. This is no longer a viable architecture in today's highly dynamic multi-tenant datacenters.

[**Q12**]How does the distributed graph of the TF relate to your data structures?

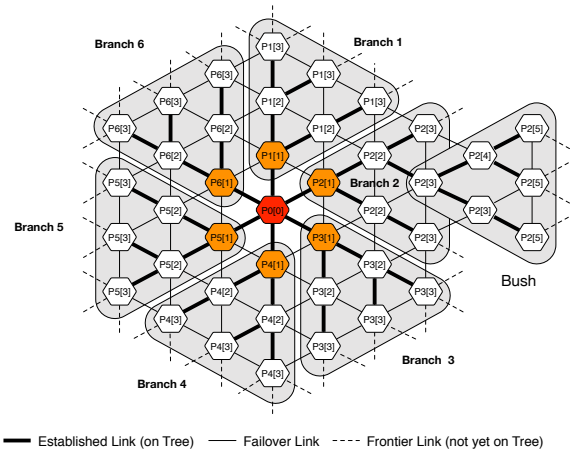
[**A12**] Our reliable distributed data structure is called the MetaData Tensor. While inspired by the mathematics of Tensors (for its conserved quantities and other nice properties), it is implemented as sets, or more accurately, as a lattice. The basic idea is that we can combine dependency graphs, which represent the in-memory versions of these lattices, with our RAFE (Reliable Address-Free Ethernet) protocols, to create a Tensor Network, capable of moving time logically backwards and forwards (for example when the system encounters errors). We use Intel's special instructions for Parallelizing Data Flow and Dependency Graphs to implement these atomically and with high performance.

[**Q13**]What's wrong with the way that addressing is done today?

[**A13**] We have an entire FAQ on this topic. But a nice simple answer to this can be found in the insightful description: The world in which IPv6 was a good design "The problem with ethernet addresses is they're assigned sequentially at the factory, so they can't be hierarchical. That means the 'bridging table' is not as nice as a modern IP routing table, which can talk about the route for a whole subnet at a time. In order to do efficient bridging, you had to remember which network bus each MAC address could be found on. And humans didn't want to configure each of those by hand, so it needed to figure itself out automatically. If you had a complex internetwork of bridges, this could get a little complicated.

[**Q14**]Why can't you just use QUIC over IP?

[**A14**] This is actually a good idea, and is hinted at in The world in which IPv6 was a good design. However, in a world (such as the TF) where we are not constrained by switch compatibility, we can do far better. We can eliminate all the unnecessary paraphernalia that has accumulated over decades of incremental band-aids to L2 and L3 addressing. Now that we understand that Mobile processes are the Killer App in networking, rather than add more band-aids, we can get rid of the lowest level addressing which evolved from a bus to a star based Ethernet, and created all this complexity and confusion in its wake. Our insights, from developments in graph theory, is that relationships one link away are very special (from an information-theoretic perspective). We exploit that in our 'Zero, One and Infinity' relative information-scaling principle:



[Q15]Why hasn't the industry solved these problems so far?

[A15] The industry's response has been to introduce Software-Defined Networking (SDN) to program networks. But SDN is (a) incremental and (b) its programmability continues to be confined to the *network control plane*; which remains under the control of network owners and operators. High-performance forwarding chips based on PISA (Protocol Independent Switch Architecture)[1] can be programmed by the P4 language. While this is a step in the right direction, it is too low a level of programmability[2], supporting the conventional notion that switches *forward* packets to *destination* addresses[3].

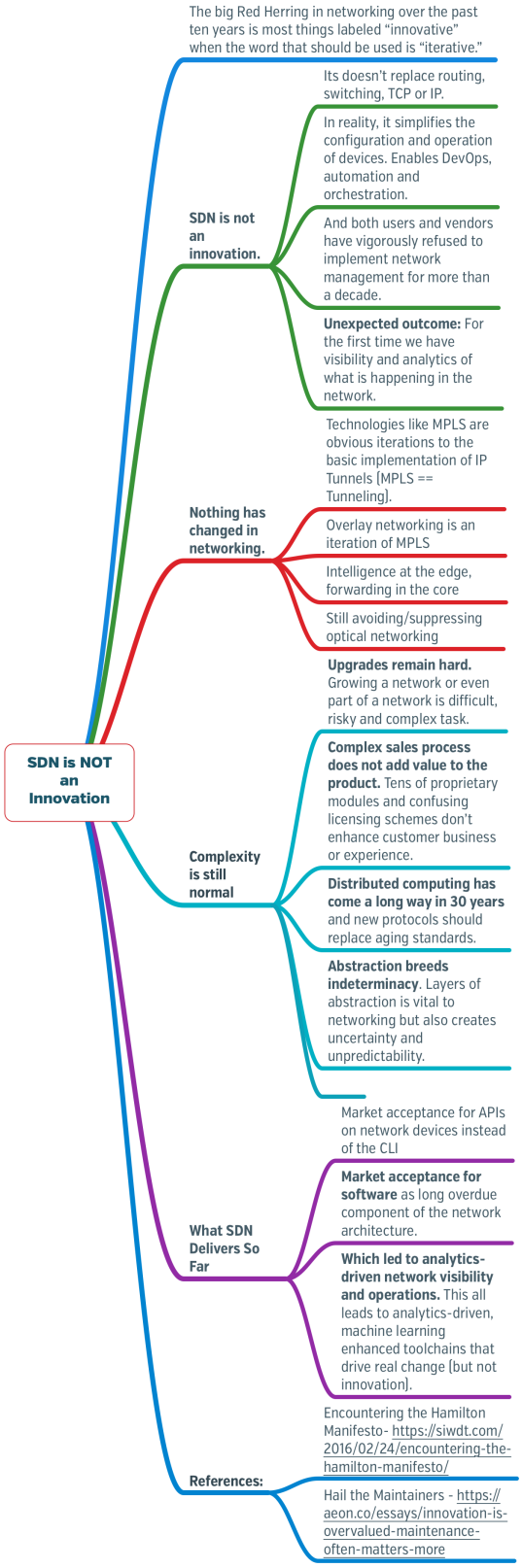
[A15] While programmable switches may be a promising approach to improve the performance and manageability of datacenters, they are still (a) under the control of the network owners and operators, and (b) limited by the low-level endpoint routing and packet forwarding paradigm of today's network engineering. What is needed to complete this revolution is to include the cells (agents on servers) as first class members of this set of devices which are allowed to route packets as well as process them.

[Q16]What about P4, eBPF and Broadcom's Network Programming Language (NPL)?

[A16] They are all operating at a different abstraction level to the Transaction Fabrix. They are extending the networking world to programmable functions (NFV), whereas we are extending the distributed systems world to networking.

[Q17]What are examples of the breakthroughs needed for the GVM to provide its benefits?

[A17] The fundamental breakthrough for distributed systems provided by the DAEDAELUS Link Protocols (DPSP), is the creation, distribution and retirement of indivisible, non-copyable tokens. We replace complex and difficult to reconcile notions of atomicity and locking in shared memory and networks, with simple and precise notions of tokens which can be accounted for across distributed systems, in the presence of all known failure hazards. We replace ambiguous isolation levels with stacked



Software Defined Networks (SDN)
From Ethereal Mind

trees on programmable data flow graphs, which can be
more easily reasoned about to manage concurrency, and be formally verified.

[**A17**] The problem we are solving is a different perspective than those normally experienced or characterized by networking professionals. This problem is seen *everywhere* by distributed systems developers. From their perspective, the problem is not characterized in the familiar concepts and terminology of bandwidth and latency, but in 'impossibility results' that prevent distributed algorithms from achieving computational goals in scale-out, microservice-based, distributed systems. Two examples of those problems are known as the CAP Theorem and the FLP Impossibility result.

[**Q18**]How does the Transaction Fabrix GVM differ from conventional Networks?

[**A18**] The following table provides a comparison and overview. See our other FAQs' for details.

Conventional – IP/port Addressing

- Static IP Address/port
- Spanning trees are built and torn down on the switches[a].
- DHCP converts 48-bit MAC address to 32/128-bit IP address
- Any to Any (A2A), End to End (E2E).
- Conflation of Naming and Address.
- Dependence on external switches/routers to build trees.
- God's-Eye-View
- Packets can be duplicated and re-ordered.
- Unreliable Multicast.

[a]Servers are always leaves, with no knowledge or control over these spanning trees.

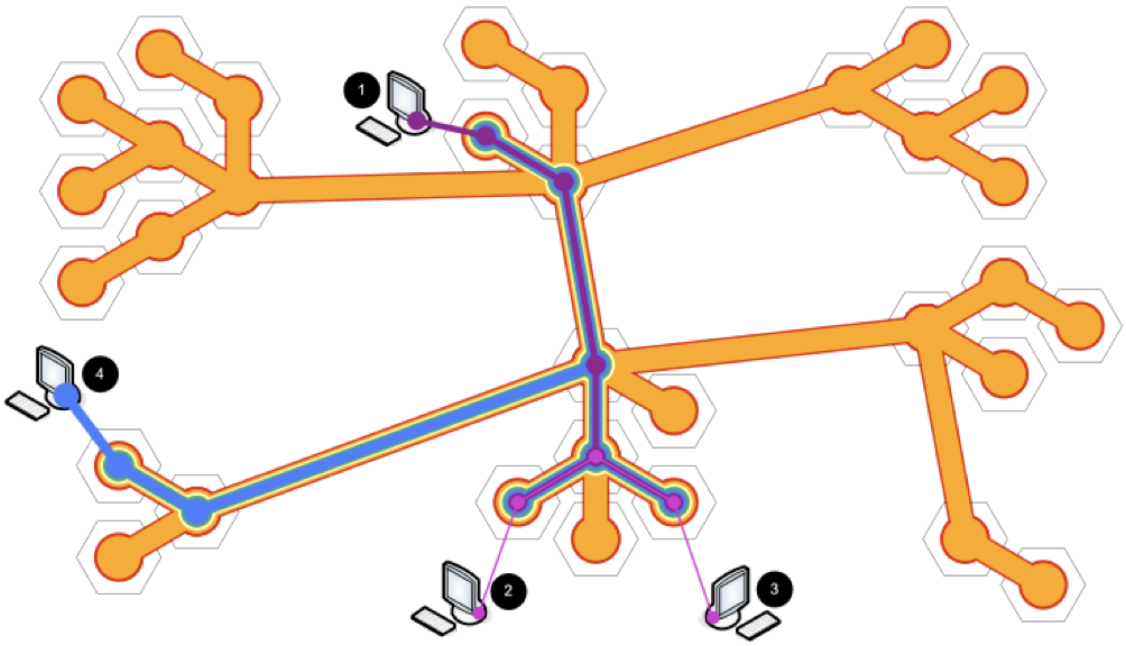
DAEDAELUS – Tree Addressing

- Dynamic Tree-based addressing
- Every cell builds a tree rooted on itself
- Self-selection of name (`treeID`) can be public key, derived from `cellID`
- Spanning Gradients
- Successively reversible protocol, composable over multiple links.
- Guaranteed in-order delivery to multiple points on tree.
- Conserved Tokens.
- A spanning tree per cell (and a spanning tree per VM / container / Lambda on that cell).
- Reliable Treecast – up to ($\delta(v)$ nodes)[a].

[a]Consensus tiles do reliable treecast only on 3, 5, 7 or 9 cells. There must be a single `Link` between the root (initiator) and each of its children 1-hop away to achieve the reliability guarantees. But this fits perfectly with the size of Paxos, Raft or Zookeper clusters, while supporting flexible membership.

[**Q19**]How do you build and stack trees?

[**A19**] Like this:



[A19] Seven stacked trees on 32 cells in the figure above (red, orange, yellow, green, blue, purple and violet).

[A19] The procedure to build the trees is as follows:

- The red tree is built as flood and prune by `discover` and `discoverd` packets. Imagine this is done from one of the center cells (it doesn't matter). The red tree is 'owned' by one cell, and spans all 32 cells[2].
- The orange tree is built 'on top' of the red tree by rootcast'ing a `stacktree` message. Because the base (red tree above) already exists, this takes the place of the `discover/discovered` on top of an existing tree. The orange tree may be built on the same root as the red tree (e.g. one of the cells in the middle.
- The yellow tree is a subset of the orange tree (graph cover over 8 cells). It can be initiated by a different member of the orange tree by rootcast'ing the stacktree message, and setting up a Routing Table (CA aware) tree. The root (of the parent tree) responds by sending a new `stacktreed` message builds the tree, not with its root pointers pointing back to itself, but pointing forward to the new root for the stacked tree. This `stacktreed` message is retired by the initiating cell, which is now the root for that stacked tree.

[Q20]Why is now the right time to unite compute and network resources?

[A20] The separation of compute and network resources is an unnatural state of affairs. The idea that each should scale independently of the other is a flawed concept. Compute, storage, CPU *and* network processing should scale together. All it does is create two silo's of technology and expertise. Two buckets of resources that have to be built, installed, maintained, managed, carried and drained separately. Two cultures of expertise, two sets of tools, two sets of policies and procedures, two sets

[2]There are other 'physical' links between each of these cells not shown in the figure because they've been pruned.

of management control – serving two classes of vendors families independently trying to create (and hide in gratuitous complexity); their 'lock-in', to subvert customers' choice in a free market. There are many reasons why we should have one type of box: a unified cell, to replace the nominally two types of boxes *switches* and *servers*, in datacenters today:

- SmartNICs can process multiple 25/50/100 Gbit/sec ports without loading the main CPU.
- SmartNICs can forward packets from port to port if they are not needed for this cell.
- There is no more need to make networks either 'Flat' (each server endpoint is equidistant in latency to every other server endpoint, with only a dumb network in between), or 'Fat' (links higher up in the Clos carry increased aggregated bandwidth, currently 400 or 800Gbit/second using link aggregation between spine switches).
- By moving from the Clos bottleneck to a free mesh, significantly more bisection bandwidth becomes available for many more 25/50/100Gbit application to application communications, eliminating the need for any one link to carry traffic from other tenants or unrelated microservice clusters into and out of the CPU.
- DAE's intrinsic reliable multicast mechanism supports fundamental distributed systems operations, such as atomic information transfer (used by distributed transactions) atomic membership and election (Consensus as a service)[3], and atomic broadcast within consensus tiles of 3-9 cells.
- The need for provably secure *confinement* (hardware enforced) to enable new forms of security.

[Q21]What are the defining characteristics of the Transaction Fabrix on which the GVM depends?

[A21] The first defining characteristic of the Transaction Fabrix (TF) is a 'free mezh'[4]. It differs from conventional Clos networks by providing cells with 8 ports instead of 1 (or 2 for simple redundancy). This is not your grandfather's mesh, hypercube or other HPC interconnect. Each cell in the TF *does not* have a globally assigned address; there is no cartesian coordinate system; even if pictures of the TF may look superficially like a rectangular array.

[A21] The second defining characteristic of the Transaction Fabrix (TF) is that there is lots of local connection, achieved by short cables connecting each node to its nearest neighbors. 8 connections provide enough density to eliminate partitions in all but the most extreme failure scenarios. These local connections are short, fast and reliable: two orders of magnitude faster than having to go through a switch ($2ns$ vs $280ns$), and many orders of magnitude more reliable when combined with our atomic `link` and successive healing protocols.

[A21] The third defining characteristic of the TF is that every cell builds its own view of the network. This is much simpler and more economical than it appears. Each cell manages its awareness of *information* about other cells on the network, by building a spanning tree rooted on itself, which only it can name (`treeid`). This is not your grandfather's Ethernet, or Internet

[3] A side benefit of reliable multicast is that it also uses less resources to perform these functions. But we don't regard saving bandwidth to be anywhere near as important as the reliability of knowing (immediately - in a purely event driven way) if an atomic operation succeeded or failed.

[4] A 'mesh-like' interconnect, without the addressing or topology assumptions of the word 'mesh' In the literature.

Protocol. There are only three levels of information: Zero, One and Infinity. Zero is what it knows about itself (under its control, within itself). One, is what it knows about its immediate (1 hop) neighbors – the only cells it can uniquely address, via the port on which it is connected. Infinity is 'everything else' – that we don't need to maintain state for, or even knowledge of its existence – the infinite sea of indistinguishable cells on the tree two hops and beyond.

[A21] The forth defining characteristic of the TF is that all failures (links or cells) are healed *locally*. This enables the groundplane of *cell trees* to not only remain stable and resilient over long periods of time and continuous change, but the *successive* nature of the failures and healing enables them to maintain in-order delivery through any number of failures and healings. This is an essential property for our ability to deliver *exactly-once* semantics for the distributed microservices that run on top, and to maintain the essential *topology* information on which applications like ML/AI depend.

[Q22]How do you achieve a 'network aware' topology with the GVM?

[A22] Network Aware topology is a GEV procedure in conventional networks, often requiring human choice and supervision for which ports on a switch are connected to which servers, and which ports on those servers. "There exists no detection service that can discover the underlying network topology in a scalable manner and expose this information to runtime libraries and users of the high performance computing systems in a convenient way" [See: Infiniband]. Some of the existing approaches have a time and space complexity of $\mathcal{O}(N_{hosts}^2)$ while others require administrative privileges to get the InfiniBand topology information which normal users of a supercomputing system will not have. Similar problems exist in Ethernet infrastructures.

[A22] In the Transaction Fabrix, all addressing is *relative* – i.e. in one of p directions from the LOV of each cell, where p is the number of connected ports. Every cell in a complex builds a spanning tree rooted on itself. Which means:

- There is only one entity (name) to keep track of: the `treeID` (not a set of endpoints that individually come, go and migrate around).
- Subtrees are *stacked* on top of each other to narrow the scope of the communication. Think of a `treeID` as a namespace. It reliably defines set membership. It's the job of the Transaction Fabrix to keep track of the liveness and membership status $\mathcal{O}(N_{cells})$, not the job of each endpoint, which scales as $\mathcal{O}(N_{hosts}^2)$ in conventional networks.
- Addressing is now by *direction* (m of p ports) which identify each 'branch' connected to a this root (self) cell's tree. The constraint on cell valency ($p = 7 \pm 2$) forces multi-hop behavior – which enables a new 'Network Defined Software' paradigm.
- Old-style endpoint addressing (using Ethernet or IP `src-dest` pairs) can be overlaid on our acyclic datacenter graph without them being aware of the completely different routing technology operating at the layer below (we could call this layer 1 routing; we are simply virtualizing conventional packets to run over the physical infrastructure – the set of black trees we call the 'groundplane').

[Q23]What are TRAPHs, and why are they important for Programmable Application Topologies?

[A23] Critical layers are missing between applications and infrastructure: a layer which contains the evolving *graph* relationships of modern microservices. A substrate that programmers can own and manage themselves[4]. This provides the missing abstraction for a programmable, and 'deterministic-on-demand', topologies as tools and resources to the application architect. For example, application programmers can program these TRAPHs (Tree gRAPHs) using a Graph Virtual Machine (GVM) to provide services such as distributed consensus, atomic broadcast, and presence management among members of a cluster or microservices set.

[A23] TRAPHs enable datacenter operators to organize *graphs* of resources, managing them on trees, enabling computing on graphs. From the perspective of different vantage points, each with least-privilage[5]. They also provide developers of microservices complete freedom (within the nodes assigned to them), to programmatically determine their topology, their evolving causal relationships, and the protocol characteristics most needed for reliable 'deterministic on demand' applications.

[A23] The advantage of using TRAPHs over a distributed network is that application developers can program their behavior instead of having to wait for permission, or suffer the externalities of the network optimizing itself without regard to the application's health. This simplifies many important use cases such as:

Logical & Virtual Segregation Planes Enable capability-based security graphs, to provide secure containment of communication environments for multi-tenant infrastructures. E.g., exchange the management of lists (ACL's and `iptables`) by replacing them with richer and more manageable `graph equations`. Virtual Segregation Planes: graph applications: erasure coding, machine learning, etc.

Coherent graph overlays where cache *heterarchies* co-exist to automatically manage the placement and eviction of caches based on request patterns. One use case would be a coherent configuration file system, which provide a unified mechanism to keep configuration files synchronized, for Docker, etc. Another would be a *coherent* `memcached`. Eliminating cascade failure incidents, like Google has seen many times spread to all regions of their Cloud Platform due race conditions to update configuration files.

Managing Infrastructure as Sets, Graphs & Tensors, *instead of* Boxes, Files & Lists. All routing is predicated on building shortest path trees, e.g. Bellman-Ford for L2, or Dijkstra at L3. The roots for these trees are in the switches, and thus under the administrative control of network owners and operators. With TRAPHs, large subgraphs, or graph-covers comprising `cells` and `links` allocated to a particular tenant, may be used by that tenant for any topology whatsoever, including allocation to sub-tenants. *Graph computing* on TRAPHs (Tree-gRAPHs) enable automatic mapping of the natural DAG relationships of distributed applications on nested datacenter infrastructure resources.

Conclusion: Allowing application developers to build and manage their own routing substrate under API control would dramatically improve the performance, efficiency and flexibility of modern infrastructures, reducing inter-tenant interference, enabling privacy, and improving manageability.

[Q24]Why have you chosen simplicity to be the most important principle for GVM?

[A24] Modern network engineering and management has become a monster of such overwhelming complexity that only decades of architectural neglect can create. Point product on point product, feature on top of feature, bug on top of bug. A Byzantine mess that increases costs, slows down business, and impedes innovation.

> "An important aspect is the terrible quality of network device software. You install a new version and a lot of things are broken. You have a perfect configuration in software simulation, but it does not work with hardware optimized devices, since the porting is never finished and barely tested by the vendor. The documentation is just a bad joke, the support forums are full with unanswered questions.
>
> There is so much bad experiences that most network engineers are reluctant to touch a working system for good reasons. You should change to a better environment with fully automated regression testing. However, when you look at the new SDN projects, the same problems come back. Everyone is rushing to include new features, and the automated testing have less than 10-20% coverage. No one wants to pay for quality. Good enough is the target... :-)"
> From: Upgrading Network Device Software.

[Q25]How does this relate this to "demand aware" networking technologies?

[A25] Demand Aware networking [[**?**] [5]] is a excellent example of the need for topologies to be dynamic on the millisecond and below scale. As with many ideas like this, there can be many variations to suit particular use cases. The GVM overcomes the need for bespoke programming of these use cases by providing a GVM Engine, which can be programmed to do graph operations using a high level graph language. The GVM Engine is build on the Interaction Engine (the P8 Programming Language instead of the P4 language), and the Slice Engine (in OAE Specifications).

- Software-Defined-Networking (SDN) provides applications with network-awareness through centralized controllers that can dynamically configure network paths. However, SDN still operates within the traditional IP addressing paradigm. It's essentially smart packet forwarding with centralized control.
- Application-Aware QoS and Routing adapts to changing network conditions and application requirements. These systems can prioritize traffic based on application needs, but they're still fundamentally reactive to congestion and packet loss.
- Adaptive and Dynamic Networking – Current SDNs use adaptive operation modes where switches request routes from controllers for packets without specific routes, but this creates latency and single points of failure.

[A25] Traditional Demand-Aware networks[**?**] react to traffic patterns and adjust routing/QoS after detecting congestion or application needs. In the GVM, the tree topology itself *embodies* the application's communication patterns. Rather than adapting to demand, the network structure is the demand pattern.

[5] Toward Demand Aware Networking

SDN controllers rely on central intelligence to make routing decisions based on a global network view. The GVM instead relies on each cell's local intelligence about its immediate neighbors, creating emergent network behavior without centralized bottlenecks.

Current systems still fundamentally reason about moving packets between IP addresses. The GVM is about maintaining relationships between entities in a graph. The network understands semantic relationships, not just packet destinations.

[**A25**] Instead of network administrators configuring demand-aware policies, GVM networks evolve organically. Related micro-services can automatically cluster on nearby tree branches. Frequently accessed data can migrate toward compute resources. The GVM goes beyond reactive demand-awareness to predictive allocation – tree branches automatically balance load without external orchestration.

Demand-Aware Networking Technologies

[**Q26**]SDN?

[**A26**] Software-Defined Networking (SDN) Current SDN provides applications with network-awareness through centralized controllers that can dynamically configure network paths. However, SDN still operates within the traditional IP addressing paradigm - it's essentially smart packet forwarding with centralized control.

Application-Aware QoS and Routing Modern SDN-based IoT networks use application-aware QoS routing algorithms that adapt to changing network conditions and application requirements. These systems can prioritize traffic based on application needs, but they're still fundamentally reactive.

Adaptive and Dynamic Networking Current SDN uses adaptive operation modes where switches request routes from controllers for packets without specific routes, but this creates latency and single points of failure.

From Reactive to Predictive Networking Architecture

Traditional Demand-Aware: Networks react to traffic patterns and adjust routing/QoS after detecting congestion or application needs

GVM: The tree topology itself embodies the application's communication patterns. Rather than adapting to demand, the network structure is the demand pattern

From Centralized Intelligence to Distributed Cognition

SDN Controllers: Central intelligence makes routing decisions based on global network view

GVM: Each cell has local intelligence about its immediate neighbors, creating emergent network behavior without centralized bottlenecks

From Address-Based to Relationship-Based Networking

Current Systems: Still fundamentally about moving packets between IP address endpoints[6].

[6] See-FAQ-Addressing

GVM: About maintaining relationships between entities in a graph - the network understands semantic relationships, not just packet destinations

Organic Network Evolution

Instead of network administrators configuring demand-aware policies, GVM networks evolve organically:

Microservice Affinity: Related services automatically cluster on nearby tree branches

Data Locality: Frequently accessed data migrates toward compute resources

Load Balancing: Tree branches automatically balance load without external orchestration

Predictive Resource Allocation GVM goes beyond reactive demand-awareness to predictive allocation:

Traffic Prophecy: Tree topology predicts communication patterns based on application structure

Elastic Branching: Tree branches grow/shrink based on anticipated rather than observed demand Preemptive Failover: Network reconfigures before failures impact applications

Contextual Network Intelligence Current demand-aware networks understand bandwidth and latency. GVM understands application semantics:

Transaction Awareness: Network knows which operations must be atomic Consistency Requirements: Different tree branches can have different consistency guarantees Security Context: Network enforces application-level security policies at the infrastructure level Comparison with Emerging Technologies

Intent-Based Networking (IBN)

IBN: "I want this application to have low latency"

GVM: "This application IS the network topology"

AI-Driven Network Management

AI Networks: Machine learning optimizes traditional packet forwarding

GVM: The network structure itself encodes the optimization through graph relationships

5G Network Slicing

5G Slicing: Creates virtual networks with different QoS characteristics

GVM Trees: Each tree branch IS a network slice with built-in isolation and QoS Real-World Demand-Aware Applications

Content Delivery Networks (CDN)

Traditional CDN: Caches content based on geographic request patterns

GVM CDN: Content automatically flows toward requestors through tree relationships

Gaming Networks

Current Gaming: Dedicated servers with complex matchmaking

GVM Gaming: Game world topology IS the network topology - players' connections reflect game relationships Financial Trading Systems

Traditional: Demand-aware networks prioritize trading traffic

GVM: Trading relationships create network paths - market makers become network hubs

The key insight is that the Graph Virtual Machine (GVM) doesn't just make networks aware of application demand - it makes the network topology itself a direct expression of application relationships. This is the difference between having a smart network that adapts to applications versus having the network BE the application's communication structure.

This represents a paradigm shift from *demand-aware* to *demand-native* networking, where the infrastructure doesn't just respond to application needs but literally embodies them in its structure.